ASSIGNMENT-3

Software Testing and Quality Assurance

MCA 2nd YEAR

Q1. What is Test Planning? Explain the main steps involved in the test planning process.

Test Planning – Meaning and Steps

1. **Understand requirements:** Study the software requirements and identify what must be tested.
2. **Define scope:** Decide which features are included and excluded from testing.
3. **Set objectives and approach:** Define the purpose, test levels, types, techniques and entry/exit criteria.
4. **Estimate resources:** Identify testers, skills, tools, hardware, software and budget required.
5. **Prepare schedule:** Estimate effort, assign responsibilities and set milestones.
6. **Identify risks and deliverables:** Record possible risks, mitigation actions, test cases, reports and other outputs.

Example: For an online shopping website, planning may include testing login, search, cart, payment and order confirmation, while assigning testers and dates.

Flowchart / Process

Study Requirements	Define Scope	Estimate Resources	Schedule&Risks	Approve Plan

Study Requirements → Define Scope → Estimate Resources → Schedule&Risks → Approve Plan

Q2. What is a Test Strategy? Explain its importance in software testing.

Test Strategy – Meaning and Importance

- **Meaning:** A test strategy is a high-level approach describing how testing will be performed across a project or organization.
- **Consistency:** It provides common methods, standards and guidelines for the testing team.
- **Risk-based focus:** It helps prioritize critical functions where failure could cause major loss.
- **Resource planning:** It guides selection of people, tools, environments and testing techniques.
- **Quality improvement:** It encourages systematic testing and early identification of defects.
- **Better communication:** It aligns developers, testers, managers and stakeholders on the overall approach.

Example: For a banking application, the strategy may emphasize security, transaction accuracy, performance and recovery testing.

Flowchart / Process

Quality Goals	Risk Analysis	Choose Approach	Tools&Resources	Consistent Testing

Quality Goals → Risk Analysis → Choose Approach → Tools&Resources → Consistent Testing

Q3. What is a Test Plan? Explain the major contents of a Test Plan.

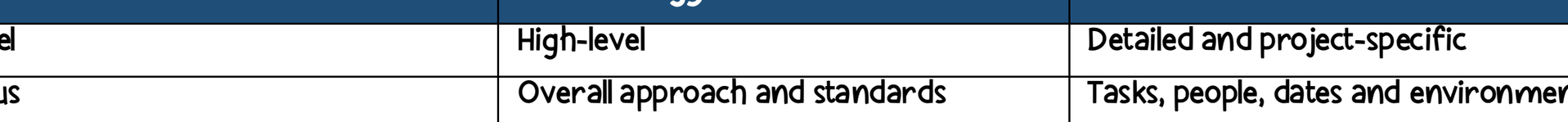
Test Plan – Meaning and Contents

- **Objective and scope:** States the purpose of testing, features to be tested and features excluded.
- **Test approach:** Describes test levels, types, techniques and the overall execution method.
- **Test environment:** Lists required devices, operating systems, browsers, databases, networks and tools.
- **Resources and responsibilities:** Identifies team members, roles, skills and ownership of tasks.

- **Schedule and estimates:** Includes activities, duration, milestones and deadlines.
- **Entry/exit criteria and risks:** Defines when testing can begin/end, known risks, assumptions and mitigation plans.

Example: A test plan for a mobile app may specify Android and iOS devices, supported versions, login test cases, assigned testers and release deadlines.

Flowchart / Process



Q4. Differentiate between a Test Strategy and a Test Plan.

Test Strategy vs Test Plan

- **Purpose:** Strategy defines the overall testing approach; plan explains how testing will be carried out for a particular project.
- **Level:** Strategy is high-level; plan is detailed and operational.
- **Coverage:** Strategy may apply to a product line or organization; plan usually targets a project, release or test cycle.
- **Contents:** Strategy covers standards, principles, risk approach and techniques; plan covers scope, tasks, roles, dates and environment.
- **Ownership:** Strategy is often set by QA leadership; a test plan is commonly prepared by a test lead or manager.
- **Change frequency:** Strategy changes less frequently; the plan may be updated as project requirements and schedules change.

Example: A company may have one security-focused testing strategy, but separate test plans for its banking app and payment gateway.

Basis	Test Strategy	Test Plan
Level	High-level	Detailed and project-specific
Focus	Overall approach and standards	Tasks, people, dates and environment
Change	Less frequent	Updated more frequently

Q5. What are the main goals and objectives of software testing? Explain.

Goals and Objectives of Software Testing

- **Detect defects:** Find errors, missing functions and unexpected behavior before release.
- **Verify requirements:** Check whether the software meets specified functional and non-functional requirements.
- **Validate user needs:** Confirm that the product is useful and works correctly for its intended users.
- **Improve quality and reliability:** Reduce failures and increase stability, usability, security and performance.
- **Reduce risk and cost:** Find important defects early, when fixing them is generally less expensive.
- **Support release decisions:** Provide evidence about quality so stakeholders can decide whether release criteria are met.

Example: Testing a payment app checks that correct amounts are transferred, invalid transactions are rejected and user information is protected.

Flowchart / Process



Q6. What are Test Phases? Explain the different phases of software testing.

Phases of Software Testing

- **Requirement analysis:** Understand requirements, identify testable conditions and clarify doubts.
- **Test planning:** Define scope, strategy, resources, estimates, schedule and risks.
- **Test design and development:** Prepare test scenarios, test cases, test data and expected results.
- **Test environment setup:** Configure hardware, software, accounts, tools and test data.
- **Test execution:** Run test cases, compare actual and expected results, and report defects.
- **Closure:** Check completion criteria, summarize results, document lessons learned and archive testware.

Example: A simple sequence is: requirements analysis → planning → test design → environment setup → execution → closure.

Flowchart / Process

Requirements	Planning	Test Design	Environment	Execution	Closure

Requirements → Planning → Test Design → Environment → Execution → Closure

Q7. What factors should be considered while developing a Test Strategy?

Factors for Developing a Test Strategy

- **Business and technical requirements:** Understand user expectations, system functions and quality needs.
- **Risk and criticality:** Prioritize areas where failure may affect safety, money, privacy or reputation.
- **Project size and complexity:** Large or integrated systems may require more levels, automation and coordination.
- **Available resources:** Consider tester skills, time, budget, tools and test environments.
- **Technology and platform:** Account for languages, architecture, devices, browsers, databases and integrations.
- **Schedule and compliance:** Consider deadlines, release frequency, industry standards, security rules and regulatory obligations.

Example: An e-commerce strategy gives high priority to checkout, payment security, inventory accuracy and peak-load performance.

Q8. Explain the resource requirements needed for software testing.

Resource Requirements for Software Testing

1. **Human resources:** Testers, test leads, automation engineers, developers and domain experts.
2. **Hardware:** Computers, mobile devices, servers, network equipment and other test devices.
3. **Software and tools:** Test management, defect tracking, automation, performance and security testing tools.
4. **Test environment:** Configured operating systems, browsers, databases, APIs, networks and staging systems.
5. **Test data:** Valid, invalid, boundary and realistic data, with sensitive information protected or masked.
6. **Time and budget:** Effort estimates, training, tool licenses, maintenance and contingency time for retesting.

Example: For a mobile banking app, resources may include Android/iOS devices, a secure staging server, masked test accounts, testers and automation tools.

Q9. What is a Test Schedule? Explain the importance of scheduling in software testing.

Test Schedule – Meaning and Importance

- **Meaning:** A test schedule maps testing activities to dates, durations, milestones and responsible people.

- **Coordination:** It coordinates testing with development, integration, deployment and business teams.
- **Resource allocation:** It helps assign people and environments without overloading them.
- **Progress tracking:** Milestones show whether activities are on time and where delays occur.
- **Risk management:** It reserves time for high-risk testing, defect fixes and regression testing.
- **Release planning:** It helps stakeholders judge whether testing can finish before the planned release.

Example: Example: test design on 1-3 November, execution on 4-8 November, defect retesting on 9-10 November and closure on 11 November.

Flowchart / Process

Plan Activities	Estimate Duration	Assign Owners	Track Progress	Adjust Schedule

Plan Activities → Estimate Duration → Assign Owners → Track Progress → Adjust Schedule

Q10. What is a Test Case? Explain the main components of a test case.

Test Case – Meaning and Components

- **Test case ID and title:** A unique identifier and a short description of the condition being checked.
- **Preconditions:** Required system state, user role, setup or data before execution.
- **Test data:** Input values, accounts or files used during the test.
- **Steps:** Clear, numbered actions the tester must perform.
- **Expected result:** The correct outcome according to requirements.
- **Actual result and status:** Observed outcome and pass/fail status; defect ID may be added if it fails.

Example: Login test case: enter a registered email and correct password, click Login, and expect the user dashboard to appear.

Q11. Explain the process of designing and writing effective test cases.

Designing Effective Test Cases

- **Study requirements:** Understand specifications, acceptance criteria and user workflows.
- **Identify test conditions:** List features, rules, error conditions and risks to verify.
- **Choose design techniques:** Use equivalence partitioning, boundary value analysis, decision tables or state transitions where suitable.
- **Prepare test data and preconditions:** Specify the exact setup and inputs required.
- **Write clear steps and expected results:** Keep each test focused, repeatable and easy to follow.
- **Review and maintain:** Check coverage, remove duplication, trace cases to requirements and update them when the software changes.

Example: For age validation, include a valid age, an invalid age, the minimum allowed age and values just below and above the boundary.

Flowchart / Process

Study Requirements	Identify Conditions	Select Technique	Write Cases	Review&Update

Study Requirements → Identify Conditions → Select Technique → Write Cases → Review&Update

Q12. What is Bug Reporting? Explain the important information included in a bug report.

Bug Reporting – Important Information

- **Bug ID and title:** A unique tracking number and a concise description of the issue.
- **Environment:** Application version, device, operating system, browser and other relevant setup.
- **Steps to reproduce:** Numbered steps that allow another person to repeat the problem.
- **Expected and actual results:** Explain what should happen and what actually happened.
- **Severity and priority:** Severity describes impact; priority indicates urgency for fixing.
- **Evidence and status:** Attach screenshots, logs or recordings, and track assignment and current status.

Example: Title: 'Checkout fails after applying a valid coupon'. Include steps, expected discount, actual error, browser/version and screenshot.

Q13. Explain the Bug Life Cycle in software testing.

Bug Life Cycle

- **New:** Tester records a newly discovered defect.
- **Assigned:** The defect is assigned to a developer or responsible team.
- **Open / In progress:** The team investigates and works on the fix.
- **Fixed:** Developer changes the code and marks the issue as fixed.
- **Retest:** Tester checks the fix in the relevant build.
- **Closed or Reopened:** If the issue is resolved, close it; if it still occurs, reopen it.

Example: A defect that still appears after a fix is marked Reopened and returned to the development team.

Flowchart / Process

New	Assigned	In Progress	Fixed	Retest	Closed

New → Assigned → In Progress → Fixed → Retest → Closed

Q14. What are Test Metrics? Explain their importance in measuring testing activities.

Test Metrics – Meaning and Importance

- **Meaning:** Test metrics are quantitative measurements used to evaluate testing progress, coverage, quality and effectiveness.
- **Progress visibility:** They show how many test cases are prepared, executed, passed or failed.
- **Quality assessment:** Defect counts and defect trends help reveal product risk and stability.
- **Coverage measurement:** Requirements or code coverage indicate which areas have been tested.
- **Planning and control:** Metrics support estimates, resource decisions and early detection of delays.
- **Improvement:** Comparing metrics across releases helps identify weak processes and improve future testing.

Example: If 80 of 100 test cases have been executed, test execution progress is 80%. This alone does not prove the software is defect-free.

Q15. Explain different types of testing metrics used in software testing.

Types of Testing Metrics

- **Test execution metrics:** Number and percentage of test cases executed, passed, failed or blocked.
- **Defect metrics:** Defects found, open defects, defect density, severity distribution and defect leakage.
- **Coverage metrics:** Requirement coverage, feature coverage or code coverage.
- **Productivity metrics:** Test cases designed/executed per person or per unit of time.
- **Efficiency metrics:** Defects detected per effort, time to detect/fix defects and retest effort.
- **Schedule and cost metrics:** Planned versus actual effort, milestone completion and testing cost.

Example: Pass rate = (Passed test cases ÷ Executed test cases) × 100. If 72 of 90 executed cases pass, pass rate = 80%.

Q16. What are Test Case Design Techniques? Explain their importance in software testing.

Test Case Design Techniques

- **Meaning:** Methods used to choose inputs, conditions and scenarios so tests are effective and systematic.
- **Equivalence partitioning:** Divides input data into groups expected to behave similarly; select representative values.

- **Boundary value analysis:** Checks values at and around limits where defects commonly occur.
- **Decision table testing:** Tests combinations of conditions and resulting actions.
- **State transition testing:** Checks valid and invalid transitions between system states.
- **Importance:** These techniques improve coverage, reduce unnecessary test cases and help uncover defects efficiently.

Example: For a field accepting values 1–100, equivalence classes include below range, valid range and above range; boundary tests include 0, 1, 100 and 101.

Q17. What is Equivalence Partitioning? Explain with a suitable example.

Equivalence Partitioning

- **Meaning:** A black-box test design technique that divides input values into classes expected to produce similar behavior.
- **Valid class:** Values accepted by the system and expected to be processed correctly.
- **Invalid classes:** Values rejected by the system, separated when they may behave differently.
- **Representative testing:** Choose one or more representative values from each class rather than testing every possible value.
- **Benefits:** Reduces test count while maintaining meaningful input coverage.
- **Limitation:** It may miss errors near limits, so it is often combined with boundary value analysis.

Example: If a field accepts integer values from 1 to 100: invalid class below 1 (e.g., 0), valid class 1–100 (e.g., 50), invalid class above 100 (e.g., 101).

Q18. What is Boundary Value Analysis? Explain with a suitable example.

Boundary Value Analysis

- **Meaning:** A black-box technique that tests values at the edges of an allowed range and just outside those edges.
- **Reason:** Errors often occur due to incorrect comparison operators, off-by-one mistakes or range handling.
- **Identify boundaries:** Find the minimum and maximum permitted values from the requirements.
- **Select test values:** Test minimum, just below minimum, just above minimum, maximum, just below maximum and just above maximum.
- **Benefits:** It finds limit-related defects with a small number of carefully chosen tests.
- **Use with other techniques:** Combine it with equivalence partitioning for stronger input coverage.

Example: For an allowed range of 1–100, test 0, 1, 2, 99, 100 and 101.

Q19. Differentiate between Equivalence Partitioning and Boundary Value Analysis.

Equivalence Partitioning vs Boundary Value Analysis

- **Main idea:** Equivalence partitioning divides inputs into behavior-based classes; boundary analysis focuses on the edges of valid ranges.
- **Selection:** Equivalence partitioning chooses representative values from each class; boundary analysis selects values at and around limits.
- **Best suited for:** Equivalence partitioning is useful for broad input categories; boundary analysis is especially useful for ordered ranges.
- **Defects targeted:** Equivalence partitioning can find class-related behavior errors; boundary analysis targets off-by-one and limit errors.
- **Number of tests:** Equivalence partitioning often reduces tests by selecting class representatives; boundary analysis adds a small set around each boundary.
- **Combined use:** Using both offers broader coverage than relying on either alone.

Example: For range 1–100, equivalence partitioning may test 0, 50 and 101; boundary value analysis tests 0, 1, 2, 99, 100 and 101.

Basis	Equivalence Partitioning	Boundary Value Analysis
Focus	Input classes	Range edges
Test values	Representative value per class	At and around boundaries
Typical use	Different categories	Minimum/maximum limits

Q20. What is Test Case Planning? Explain the steps involved in test case planning.

Test Case Planning – Steps

- **Review requirements:** Study functional and non-functional requirements and acceptance criteria.
- **Define test scope:** Identify features, workflows, platforms and risks to cover.
- **Identify scenarios:** Break requirements into testable situations, including positive and negative cases.
- **Select techniques and data:** Choose suitable design methods and prepare required input data.
- **Estimate and prioritize:** Estimate effort and prioritize cases by risk, business importance and dependencies.
- **Document and review:** Record cases, expected results, traceability and execution order; review and update them as needed.

Example: For a registration form, plan cases for valid details, missing required fields, invalid email, password rules and duplicate email.

Flowchart / Process



Review Requirements → Define Scope → List Scenarios → Design Cases&Data → Prioritize → Review